AAI520_Group7_Final_Light

October 10, 2026

AI Investment Committee

AAI-520 Final Team Project, Group 7: Michael Valderrama, Guna Pasupathy, Mina Habib

Repository: https://github.com/ibucketbranch/AAI-520-Final-Project

Our project is a small "investment committee" made of AI agents. You give it a stock symbol. A planner decides what to research, three tools collect earnings, news, and market data, a Bull analyst and a Bear analyst argue from that evidence, an evaluator checks every claim against the evidence it cites, and the committee writes a short memo with a verdict. A memory note from each run is read by the planner on the next run.

This is a class project to show agent design and workflow patterns. **It is not financial advice.**

This is the **light** version. It calls the functions in our package and explains the results. The code itself lives in `src/investment_committee/` in the repository. A full version of this notebook prints that code inline after every section.

How to run it

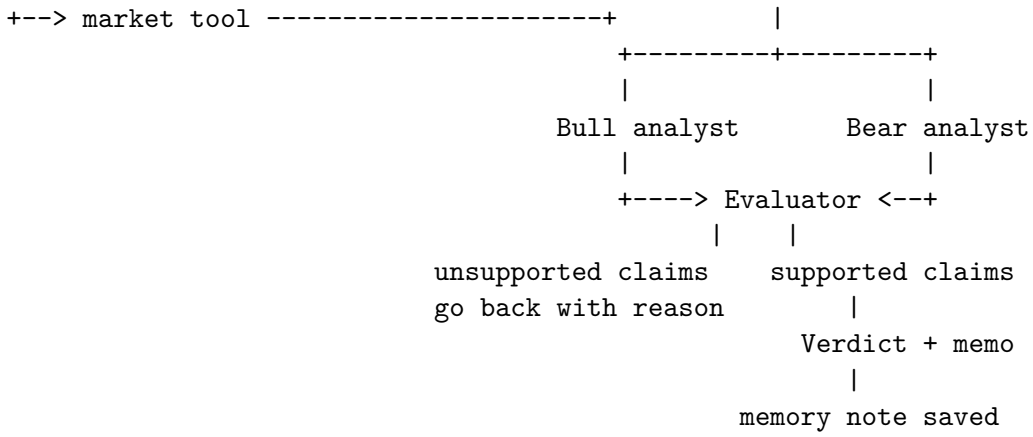
1. From the repo root: `pip install -r requirements.txt` (or `make setup`). The setup cell adds `src/` to Python's path, so the package works even if you skip `pip install -e .`.
2. Optional: copy `.env.example` to `.env` and add `OPENAI_API_KEY` (and `NEWSAPI_KEY` for news). Without keys everything still runs: the planner, analysts, and evaluator fall back to their rule-based versions.
3. `LIVE = True` (below) uses real market and news data, so it needs internet. `LIVE = False` uses the small sample files in `tests/fixtures/`.

The outputs saved in this notebook come from one live run with an OpenAI key. Live data and model answers change from run to run, so a re-run will show different numbers.

1. Agent design and workflows

The committee has one job: give a short, evidence-backed opinion on a stock, and be honest about what it could not check.

```
stock symbol
     |
  Planner  <-------------- memory note from the last run
     |
     +--> earnings tool --------------------+
     +--> news tool --> news chain --> gate +--> evidence pool (SQLite)
```



ChatGPT was used for formatting this text diagram

The agents

Agent	What it does
Planner	Picks which sources to research and in what order. Reads the last run's memory note.
Tools (earnings, news, market)	Collect real data with yfinance and NewsAPI and turn it into `Evidence` records.
Bull and Bear analysts	Each writes 2 to 5 claims for its side. Every claim must cite evidence ids from the pool.
Evaluator	Checks each claim against the evidence it cites and gives a reason when a claim is not supported.
Chair (verdict)	Reads only the supported claims and picks bull, bear, or none.

The three workflow patterns (from Anthropic, 2024)

1. **Prompt chaining:** the news chain runs ingest, preprocess, classify, gate, extract, and summarize, one step after another (section 3).
2. **Routing:** each planned step is sent to its own specialist tool (section 2).
3. **Evaluator-optimizer:** the analysts write claims, the evaluator judges them, and rejected claims go back to be rewritten (section 5).

Design choices we made on purpose

- **Everything cites evidence.** A claim without a valid evidence id is rejected, so the memo can always be traced back to data.
- **Every agent has a fallback.** No API key, a failed call, or a bad model answer drops back to a rule-based version and the run records why. A broken data source does not stop the run.
- **One database as the handoff.** Each stage reads the previous stage's results from SQLite by run id. This is also what makes the with/without-review experiment fair (section 7).

Setup

This cell finds the repo folder, makes the package in `src/` importable, checks that the spaCy English model is installed (the news chain needs it for named entities), and loads the API keys from `.env` if there is one. It also starts a fresh database file just for this notebook.

[1]:
```
import subprocess
import sys
from contextlib import closing
from pathlib import Path

import pandas as pd
import spacy
from IPython.display import display

# Find the repo root (the folder with pyproject.toml), so paths work from any␣
 ↪folder.
ROOT = Path.cwd()
while not (ROOT / "pyproject.toml").exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent

# Use the package straight from src/, so this works even without "pip install␣
 ↪-e .".
sys.path.insert(0, str(ROOT / "src"))

# The news chain uses spaCy's small English model. Download it once if it is␣
 ↪missing.
# If the download fails (no internet), use a blank pipeline: the chain still␣
 ↪runs,
# it just finds no named entities.
try:
    nlp = spacy.load("en_core_web_sm")
except OSError:
    try:
        subprocess.run([sys.executable, "-m", "spacy", "download",␣
 ↪"en_core_web_sm"], check=True)
        nlp = spacy.load("en_core_web_sm")
    except (subprocess.CalledProcessError, OSError):
        print("Could not get the spaCy model, so entity extraction will find␣
 ↪nothing.")
        nlp = spacy.blank("en")

from investment_committee.config import load_settings

settings = load_settings(str(ROOT / ".env"))
HAVE_OPENAI = bool(settings.values.get("OPENAI_API_KEY"))
HAVE_NEWSAPI = bool(settings.values.get("NEWSAPI_KEY"))
```

```
LIVE = True

# "openai" uses the models; "template" is the rule-based fallback used without␣
 ↪a key.
PROVIDER = "openai" if HAVE_OPENAI else "template"

# A fresh database just for this notebook, so old runs do not mix in.
DB_PATH = ROOT / "data" / "final_notebook.db"
DB_PATH.parent.mkdir(exist_ok=True)
if DB_PATH.exists():
    DB_PATH.unlink()

print("Repo folder:     ", ROOT.name)
print("Live data:       ", LIVE)
print("OpenAI key found:", HAVE_OPENAI, "->", PROVIDER)
print("NewsAPI key found:", HAVE_NEWSAPI)
```

```
Repo folder:      AAI-520-Final-Project
Live data:        True
OpenAI key found: True -> openai
NewsAPI key found: True
```

[2]:
```
import inspect

from investment_committee.workflows.chaining import clean_text

pd.set_option("display.max_colwidth", 200)


def plain(value):
    """Make text plain ASCII for display (news and model text can have curly␣
 ↪quotes)."""
    if isinstance(value, str):
        return "\n".join(clean_text(line) for line in value.splitlines())
    return value


def table(rows):
    """Show a list of dicts as a table, with plain text."""
    frame = pd.DataFrame(rows)
    if not frame.empty:
        frame = frame.map(plain)
    return frame


def show_code(*functions):
```

```
    """Print the real source code of package functions, with the file name."""
    for function in functions:
        path = inspect.getsourcefile(function)
        short = path.split("src/")[-1] if "src/" in path else path
        lines, start = inspect.getsourcelines(function)
        print(f"# ---- {short}, line {start}")
        print("".join(lines))
```

[3]:
```
import json

from investment_committee.models import Evidence
from investment_committee.tools.earnings import gather_earnings
from investment_committee.tools.market import gather_market
from investment_committee.tools.news import gather_news
from investment_committee.workflows.chaining import RawArticle,␣
 ↪processed_to_evidence, run_chain

SAMPLE_EVIDENCE = json.loads((ROOT / "tests/fixtures/sample_evidence.json").
 ↪read_text())
SAMPLE_ARTICLES = json.loads((ROOT / "tests/fixtures/sample_articles.json").
 ↪read_text())


def sample_gatherer(source):
    """Offline stand-in for a tool: the sample evidence rows for one source."""

    def gather(run):
        rows = []
        for item in SAMPLE_EVIDENCE["evidence"]:
            if item["source"] == source:
                rows.append(
                    Evidence(
                        run_id=run.run_id,
                        symbol=run.symbol,
                        source=source,
                        title=item["title"],
                        content=item["content"],
                        as_of=item["as_of"],
                        reference="sample data",
                    )
                )
        return rows

    return gather


def sample_news(run):
```

```
    """Offline stand-in for the news tool: the saved sample articles for this␣
 ↪stock."""
    articles = []
    for item in SAMPLE_ARTICLES:
        if item["symbol"] == run.symbol:
            articles.append(RawArticle(**item))
    return articles


def chained_news(run):
    """News tool plus the news chain, so news comes back as Evidence."""
    articles = gather_news(run) if LIVE else sample_news(run)
    return processed_to_evidence(run, run_chain(articles, nlp))


if LIVE:
    TOOLS = {"earnings": gather_earnings, "news": chained_news, "market":␣
 ↪gather_market}
else:
    TOOLS = {
        "earnings": sample_gatherer("earnings"),
        "news": chained_news,
        "market": sample_gatherer("market"),
    }
```

2. Planner and dynamic tool use

Rubric items: the agent plans its research steps, uses tools dynamically, and routes work to specialists.

2.1 The planner makes a plan

We have two planners. The rule-based one always gives the same three steps in a fixed order. The OpenAI planner asks a model to choose one to three sources and explain why. The model's answer has to fit a fixed format (a list of steps, each with a source and a reason), so a bad answer is rejected instead of breaking the run. If there is no key or the call fails, it falls back to the rule-based plan and records why.

[4]:
```
from investment_committee.agents.planner import plan_research
from investment_committee.models import Run

SYMBOL = "AAPL"


def plan_table(plan):
    rows = []
    for number, task in enumerate(plan.tasks, start=1):
```

```
        rows.append({"step": number, "source": task.source, "reason": task.
 ↪reason})
    return table(rows)


demo_run = Run(symbol=SYMBOL)
rules = plan_research(SYMBOL, demo_run.run_id, provider="rules")
print("Rule-based plan:")
display(plan_table(rules.plan))

model_plan = plan_research(SYMBOL, demo_run.run_id, provider="openai",␣
 ↪settings=settings)
print("Planner asked for:", model_plan.provider_requested, "| used:",␣
 ↪model_plan.provider_used)
if model_plan.fallback_reason:
    print("Fell back because:", model_plan.fallback_reason)
if model_plan.model_call:
    call = model_plan.model_call
    print("Model:", call.model, "| tokens in/out:", call.prompt_tokens, "/",␣
 ↪call.completion_tokens)
plan_table(model_plan.plan)
```

```
Rule-based plan:
```

	step	source	reason
0	1	earnings	Collect the latest quarterly performance.
1	2	news	Collect recent events that may affect the company.
2	3	market	Collect recent price history and market indicators.

```
Planner asked for: openai | used: openai
Model: gpt-6-luna | tokens in/out: 207 / 99
```

[4]:

	step	source	reason
0	1	earnings	Review Apple's latest results, guidance, and management commentary.
1	2	market	Check AAPL's recent price performance and trading trends.
2	3	news	Identify recent company or industry developments affecting the outlook.

The plan also reacts to memory. If the last run for a stock left a note saying a source was missing, that source moves to the front. Here we hand the planner a pretend note. Section 6 does this for real, with a note written by an actual run.

[5]:
```
from investment_committee.models import MemoryNote

pretend_note = MemoryNote(
```

```
    run_id="earlier-run",
    symbol=SYMBOL,
    confidence=0.4,
    open_gaps=["no news evidence in the pool"],
    lesson="Check the news first next time.",
)
with_note = plan_research(SYMBOL, demo_run.run_id, provider="rules",␣
 ↪previous_note=pretend_note)
plan_table(with_note.plan)
```

[5]:

	step	source	reason
0	1	news	Revisit the news gap reported by the previous run.
1	2	earnings	Collect the latest quarterly performance.
2	3	market	Collect recent price history and market indicators.

2.2 Routing: each step goes to its own tool

The router has a simple map from source to tool. To show that it follows the plan, we give it a plan with only a market step and record which tools get called. Only the market tool should run.

[6]:
```
from investment_committee.agents.planner import ResearchPlan, ResearchTask
from investment_committee.workflows.routing import default_gatherers, route_plan

for source, tool in default_gatherers().items():
    print(f"{source:9} -> {tool.__module__}.{tool.__name__}")

called = []


def record_calls(source, tool):
    def gather(run):
        called.append(source)
        return tool(run)

    return gather


market_only = ResearchPlan(
    symbol=SYMBOL, tasks=[ResearchTask(source="market", reason="Only check the␣
 ↪price.")]
)
watched = {source: record_calls(source, tool) for source, tool in TOOLS.items()}
routed = route_plan(Run(symbol=SYMBOL), market_only, gatherers=watched)
print()
print("Tools called:", called)
```

```
earnings  -> investment_committee.tools.earnings.gather_earnings
news      -> investment_committee.tools.news.gather_news
```

```
market    -> investment_committee.tools.market.gather_market


Tools called: ['market']
```

2.3 The tools return real data

Now we route the full plan, so all three tools run. Earnings and market data come from yfinance. News comes from yfinance, then NewsAPI, then yfinance search headlines, whichever returns articles first. (Yahoo's ticker news feed stopped working in early October 2026, which is why the news tool has fallbacks.) News is passed through the news chain from section 3 before it becomes evidence.

[7]:
```
tool_run = Run(symbol=SYMBOL)
full_plan = plan_research(SYMBOL, tool_run.run_id, provider="rules").plan
gathered = route_plan(tool_run, full_plan, gatherers=TOOLS)


def evidence_table(items):
    rows = []
    for item in items:
        rows.append(
            {
                "source": item.source,
                "title": item.title,
                "content": item.content,
                "as of": item.as_of,
            }
        )
    return table(rows)


for source, items in gathered.items():
    print(f"{source}: {len(items)} evidence records")
evidence_table(gathered["earnings"] + gathered["market"])
```

```
earnings: 3 evidence records
news: 4 evidence records
market: 3 evidence records
```

[7]:

	source	title	content	\
0	earnings	Quarterly revenue		
1	earnings	Quarterly net income		
2	earnings	Diluted EPS		
3	market	Latest close		
4	market	RSI(14)		
5	market	MACD(12,26,9)		

```
0    Quarterly revenue was $109.42 billion for the quarter ended 2026-06-30.
1  Quarterly net income was $29.79 billion for the quarter ended 2026-06-30.
2          Diluted EPS was $2.02 per share for the quarter ended 2026-06-30.
3                                                    AAPL closed at $336.64.
4                                                  The 14-day RSI was 56.1.
5                                  MACD was 3.43; the signal line was 3.87.

        as of
0  2026-06-30
1  2026-06-30
2  2026-06-30
3  2026-10-09
4  2026-10-09
5  2026-10-09
```

[8]:
```
evidence_table(gathered["news"]).head(6)
```

[8]:
```
  source  \
0   news
1   news
2   news
3   news

                          title  \
0  Apple Shares Fall Following Report of Cut In iPhone 18 Pro Orders In Face of
Lower Demand Due to Price Hike
1                            Google's next TV Streamer sounds like a massive
upgrade - Apple should be worried
2                                Apple marks original Apple TV 4K as 'vintage,'
days before expected successor
3                                                 Apple Scales Back iPhone 18
Orders as Price Hike Cools Demand

                                     content  \
0  Apple shares fell more than 2.5% in early trading today following a Nikkei
Asia's report that the company is cutting iPhone 18 Pro component orders thanks
to weaker-than-expected demand. Read More…
1  Google's next TV Streamer could feature built-in microphones, presence
detection, faster performance, and a redesigned remote. Google's next TV
Streamer will reportedly feature a slightly narrower…
2  Apple today added the first-generation Apple TV 4K, released in 2017, to the
vintage products list on its website, four days before a new… The post Apple
marks original Apple TV 4K as 'vintage,'…
3  Apple has reduced its orders of components for the latest iPhones for October
by at least 15%, Reuters reported Friday, citing a paywalled article by Nikkei
Asia. The cuts include components for A…
```

```
        as of
0  2026-10-09
1  2026-10-09
2  2026-10-09
3  2026-10-09
```

2.4 The whole thing in one call

`run_research` does all of the above: read the last memory note, plan, route, run the tools, and save the run and its evidence to SQLite. (We pass in our `TOOLS`, whose news tool runs the news chain with the `nlp` pipeline loaded in setup.) If one source has no data (for example an ETF has no earnings), the run still finishes and says which source was missing. This saved run is the evidence pool the next sections argue from.

[9]:
```
from investment_committee.storage import load_evidence, open_db
from investment_committee.workflows.runner import run_research

snapshot = run_research(
    SYMBOL,
    gatherers=TOOLS,
    database_path=DB_PATH,
    planner_provider="openai",
)
MAIN_RUN = snapshot.run.run_id

print("Run id:         ", MAIN_RUN)
print("Planner used:   ", snapshot.planner_provider_used, snapshot.planner_model␣
 ↪or "")
print("Plan:           ", [task.source for task in snapshot.plan.tasks])
print("Saved evidence:", snapshot.saved_evidence_count)
for source, message in snapshot.gather_errors.items():
    print(f"Warning ({source}):", plain(message))

with closing(open_db(DB_PATH)) as conn:
    POOL = load_evidence(conn, MAIN_RUN)
table(
    [
        {"source": source, "records": sum(1 for e in POOL if e.source ==␣
 ↪source)}
        for source in ("earnings", "news", "market")
    ]
)
```

```
Run id:         4b0dce2d1da0
Planner used:   openai gpt-6-luna
Plan:           ['earnings', 'market', 'news']
Saved evidence: 10
```

[9]:

	source	records
0	earnings	3
1	news	4
2	market	3

3. News chain: ingest, preprocess, classify, gate, extract, summarize

Rubric item: prompt chaining. Each step's output is the next step's input.

We show the chain on the saved sample articles first, because they include tricky cases on purpose: an exact duplicate, an empty article, HTML tags, and curly quotes.

Step 1: ingest

[10]:
```
from investment_committee.workflows.chaining import (
    classification_gate,
    classify,
    extract,
    ingest,
    preprocess,
    summarize,
)

articles = ingest(ROOT / "tests/fixtures/sample_articles.json")
table(
    [
        {"id": a.article_id, "symbol": a.symbol, "title": a.title, "body␣
 ↪length": len(a.body)}
        for a in articles
    ]
)
```

[10]:

	id	symbol	\
0	a-001	AAPL	
1	a-002	AAPL	
2	a-003	AAPL	
3	t-001	TSLA	
4	t-002	TSLA	
5	t-003	TSLA	
6	n-001	NFLX	
7	n-002	NFLX	

	title	body length
0	Apple beats quarterly earnings expectations	221
1	Apple beats quarterly earnings expectations	227
2	Apple unveils new on-device translation feature	182
3	Regulator opens investigation into Tesla driver-assist claims	197
4	Analyst cuts Tesla price target on margin pressure	144

```
5                  Tesla energy division reports record quarter          199
6                Netflix cafe documentary draws festival praise          165
7                     Netflix placeholder item with empty body           10
```

Step 2: preprocess

Remove HTML, turn curly quotes into plain ones, collapse spaces, and drop empty articles and exact duplicates.

[11]:
```
kept = preprocess(articles)
kept_ids = [article.article_id for article, _ in kept]
print("In:", len(articles), "| out:", len(kept))
print("Dropped:", [a.article_id for a in articles if a.article_id not in␣
 ↪kept_ids])
print()
# The raw text has a curly apostrophe; backslashreplace prints it as \u2019 so␣
 ↪we can see it.
raw = articles[0].body.encode("ascii", "backslashreplace").decode()
print("Before:", raw[60:190])
print("After: ", kept[0][1][60:175])
```

```
In: 8 | out: 6
Dropped: ['a-002', 'n-002']

Before: analyst expectations.<br /><br />Chief executive Tim Cook said the
company\u2019s services business carried the quarter. Guidance
After:  analyst expectations. Chief executive Tim Cook said the company's
services business carried the quarter. Guidance f
```

`a-002` is the same story as `a-001` with extra spaces, and `n-002` has an empty body, so both are dropped. After cleaning, the `<br />` tags are gone and the curly apostrophe (`\u2019`) is a plain `'`.

Steps 3 and 4: classify, then the gate

`classify` gives each article a category from keywords. The **gate** comes right after it: only categories that can move a stock price (earnings, product, legal, market) are allowed into the evidence pool. Articles classified as `general` stop here.

[12]:
```
rows = []
for article, text in kept:
    category = classify(text)
    accepted, reason = classification_gate(category)
    rows.append(
        {
            "id": article.article_id,
            "title": article.title,
            "category": category,
            "accepted": accepted,
            "gate reason": reason,
```

```
        }
    )
table(rows)
```

[12]:

	id	title	category	accepted	gate reason
0	a-001	Apple beats quarterly earnings expectations	earnings	True	accepted classification: earnings
1	a-003	Apple unveils new on-device translation feature	product	True	accepted classification: product
2	t-001	Regulator opens investigation into Tesla driver-assist claims	legal	True	accepted classification: legal
3	t-002	Analyst cuts Tesla price target on margin pressure	market	True	accepted classification: market
4	t-003	Tesla energy division reports record quarter	earnings	True	accepted classification: earnings
5	n-001	Netflix cafe documentary draws festival praise	general	False	rejected classification: general

Steps 5 and 6: extract and summarize

`extract` finds named entities (companies, people, money, dates) with spaCy. `summarize` keeps the first two sentences.

[13]:
```
first_text = kept[0][1]
entities = extract(first_text, nlp)
display(table([{"entity": e.text, "label": e.label} for e in entities]))
print("Summary:", summarize(first_text))
```

	entity	label
0	Apple	ORG
1	quarterly	DATE
2	$91.2 billion	MONEY
3	Tim Cook	PERSON
4	the quarter	DATE
5	the December quarter	DATE

```
Summary: Apple reported quarterly revenue of $91.2 billion, ahead of analyst
expectations. Chief executive Tim Cook said the company's services business
carried the quarter.
```

The whole chain, and the handoff to evidence

`run_chain` runs all the steps. `processed_to_evidence` turns only the **accepted** articles into shared `Evidence` records. Rejected articles stay in the chain output so we can count them, but they never reach the analysts. Evidence belongs to one stock, so here we only hand off the Apple articles.

[14]:
```
apple_articles = [a for a in articles if a.symbol == "AAPL"]
processed = run_chain(apple_articles, nlp)
sample_run = Run(symbol="AAPL")
news_evidence = processed_to_evidence(sample_run, processed)
print(
    "Processed:",
    len(processed),
    "| accepted:",
    sum(p.accepted for p in processed),
    "| became evidence:",
    len(news_evidence),
)
evidence_table(news_evidence)
```

```
Processed: 2 | accepted: 2 | became evidence: 2
```

[14]:

	source	title	content	as of
0	news	Apple beats quarterly earnings expectations	Apple reported quarterly revenue of $91.2 billion, ahead of analyst expectations. Chief executive Tim Cook said the company's services business carried the quarter.	2026-09-08
1	news	Apple unveils new on-device translation feature	At a press event in Cupertino, Apple announced a product update that adds on-device translation. The launch is planned for October, and the feature runs without a network connection.	2026-09-09

The same chain on live news. These are the live articles for our stock. Real news has more noise than the samples, so the gate matters more here.

[15]:
```
live_articles = gather_news(Run(symbol=SYMBOL)) if LIVE else␣
 ↪sample_news(Run(symbol=SYMBOL))
live_processed = run_chain(live_articles, nlp)
print(
    "Live articles:",
    len(live_articles),
    "| accepted by the gate:",
    sum(p.accepted for p in live_processed),
)
table(
    [
        {"title": p.article.title, "category": p.category, "accepted": p.
 ↪accepted}
```

```
        for p in live_processed
    ]
)
```

```
Live articles: 10 | accepted by the gate: 4
```

[15]:

```
                                                 title  \
0  EBAY : Extra 20% off all EXCELLENT CERTIFIED Refurb SAMSUNG AND APPLE Phones
at link with 1-2 yr Allstate Warranty (4 replies)
1
Apple TV: Green Book $4.99
2                              Best Buy updates massive Techtober Prime Day sale
with new deals: Apple, Android, smart home, more
3                                                                    Apple Stores
Will Add Your Name to iPhone Duo's Startup Screen
4                                                  iPhone Duo buyers to get a
special surprise at U.S. Apple Stores, Gurman says
5                                  5 Best New Movies And Shows To Stream This
Weekend On Netflix, Prime Video, Apple TV And More
6                    Apple Shares Fall Following Report of Cut In iPhone 18 Pro
Orders In Face of Lower Demand Due to Price Hike
7                                                Google's next TV Streamer sounds
like a massive upgrade - Apple should be worried
8                                                  Apple marks original Apple
TV 4K as 'vintage,' days before expected successor
9                                                                    Apple Scales
Back iPhone 18 Orders as Price Hike Cools Demand

  category  accepted
0  general     False
1  general     False
2  general     False
3  general     False
4  general     False
5  general     False
6   market      True
7  product      True
8  product      True
9  product      True
```

4. Bull and Bear over the evidence pool

Rubric item: agents that use the evidence to argue a position.

Both analysts use one shared drafting engine with opposite instructions. Each one reads the evidence pool from section 2.4 and must return 2 to 5 claims, and **every claim has to cite evidence ids from the pool**. The prompt also tells the model that the evidence text is data, not instructions (news text could contain anything).

If a claim cites an id that is not in the pool, we keep it on purpose. Throwing it away would hide the mistake. The evaluator in section 5 rejects it with a reason instead.

[16]:
```
from investment_committee.agents.bear import draft_bear_claims_model
from investment_committee.agents.bull import draft_bull_claims_model

titles = {item.evidence_id: item.title for item in POOL}


def claims_table(claims):
    rows = []
    for claim in claims:
        cited = [titles.get(eid, "NOT IN POOL: " + eid) for eid in claim.
 ↪evidence_ids]
        rows.append({"side": claim.side, "claim": claim.text, "cites": "; ".
 ↪join(cited)})
    return table(rows)


bull = draft_bull_claims_model(POOL, settings, provider=PROVIDER)
bear = draft_bear_claims_model(POOL, settings, provider=PROVIDER)
for name, result in (("Bull", bull), ("Bear", bear)):
    print(
        f"{name}: {len(result.claims)} claims, provider used: {result.
 ↪provider_used}",
        f"| unknown ids cited: {result.dropped_ids or 'none'}",
    )
    if result.fallback_reason:
        print("  fell back because:", plain(result.fallback_reason))
claims_table(bull.claims + bear.claims)
```

```
Bull: 2 claims, provider used: openai | unknown ids cited: none
Bear: 3 claims, provider used: openai | unknown ids cited: none
```

[16]:

	side	claim	cites
0	bull	Apple's latest quarter demonstrates substantial business scale and profitability: it generated $109.42 billion in revenue and $29.79 billion in net income, with diluted EPS of $2.02. That earnings…	AAPL-Q1-2026-EARNINGS
1	bull	A 14-day RSI of 56.1 is consistent with a relatively balanced, slightly positive momentum reading rather than an oversold setup, offering no clear sign of excessive short-term bullishness.	AAPL-RSI-14D
2	bear		
3	bear		
4	bear		

```
2  The clearest near-term risk is weaker iPhone demand: Apple reportedly cut
October component orders by at least 15%, including for the iPhone 18 Pro and
Pro Max, after a price hike cooled demand. T…
3                                          The reported demand concerns have
already weighed on sentiment, with shares falling more than 2.5% in early
trading after news of the iPhone 18 Pro order cuts.
4               Near-term momentum also looks less favorable: MACD at 3.43 was
below its 3.87 signal line, a bearish technical signal, though this indicator
alone does not establish a longer-term trend.

                         cites
0                                                          Quarterly revenue;
Quarterly net income; Diluted EPS
1
RSI(14)
2                                                 Apple Scales Back iPhone 18
Orders as Price Hike Cools Demand
3  Apple Shares Fall Following Report of Cut In iPhone 18 Pro Orders In Face of
Lower Demand Due to Price Hike
4
MACD(12,26,9)
```

5. Evaluator-optimizer loop and the verdict

Rubric items: the agent self-reflects on the quality of its output, and the evaluator-optimizer workflow.

The loop works like this:

1. Bull and Bear draft claims (section 4).
2. The **evaluator** checks every claim in two gates. The rules gate rejects any claim that cites evidence not in the pool. The model gate reads each remaining claim next to the text of the evidence it cites and decides if that text really supports it, with a one-sentence reason.
3. Rejected claims go **back to their analyst with the reason**, and the analyst writes a replacement. Supported claims are kept.
4. This repeats until everything is supported or we hit the round limit (2 rounds).
5. A final call reads **only the supported claims** and picks bull, bear, or none. We do not just count claims, because then the side that writes more small claims would win.

We run the committee as a new run that points to the evidence run from section 2.4 (its "parent"), so it argues from exactly that pool.

[17]:
```
from investment_committee.storage import save_run
from investment_committee.workflows.evaluator_optimizer import␣
 ↪run_committee_from_storage

with closing(open_db(DB_PATH)) as conn:
    committee_run = Run(symbol=SYMBOL, with_review=True, parent_run_id=MAIN_RUN)
    save_run(conn, committee_run)
```

```
    result = run_committee_from_storage(conn, committee_run.run_id, settings,␣
 ↪provider=PROVIDER)

print("Rounds:", result.rounds, "| verdict:", result.verdict)
print("Fallbacks:", result.fallbacks or "none")
```

```
Rounds: 2 | verdict: bear
Fallbacks: none
```

Every claim the evaluator saw, round by round. Revision 0 is the first draft. Revision 1 claims are rewrites of claims that were rejected in round 1.

[18]:
```
rows = []
for claim in result.all_claims:
    rows.append(
        {
            "revision": claim.revision,
            "side": claim.side,
            "supported": claim.supported,
            "evaluator reason": claim.reason,
            "claim": claim.text,
        }
    )
table(rows)
```

[18]:

	revision	side	supported	evaluator reason	claim
0	0	bull	True	The cited evidence confirms the stated revenue, net income, and diluted EPS for the quarter, while offering no year-over-year comparison; describing these results as a strong financial base is a r…	Revenue, net income, and diluted EPS for the quarter
1	0	bull	False	The evidence reports an RSI of 56.1 but does not establish that this is above the midpoint, mildly constructive, or below an overbought threshold.	RSI of 56.1 is above the midpoint, mildly constructive, but below an overbought threshold
2	0	bear	True	The cited reports describe at least 15% component-order cuts amid weaker demand following a price hike and a more-than-2.5% share decline, while not establishing the eventual impact on results.	Reports of at least 15% component-order cuts amid weaker demand following a price hike and a more-than-2.5% share decline
3	0	bear	True	The evidence states that MACD was 3.43 and the signal line was 3.87, so MACD was below its signal line, which can be a bearish momentum signal.	MACD was 3.43 and the signal line was 3.87, so MACD was below its signal line, which can be a bearish momentum signal
4	1	bull	False	The evidence reports an RSI of 56.1 but does not establish that this is above the midpoint, mildly constructive, or below an overbought threshold.	RSI of 56.1 is above the midpoint, mildly constructive, but below an overbought threshold
5	0	bull	True	The cited evidence confirms the stated revenue, net income, and diluted EPS for the quarter, while offering no year-over-year comparison; describing these results as a strong financial base is a r…	Revenue, net income, and diluted EPS for the quarter
6	0	bear	True	The cited reports describe at least 15% component-order cuts amid weaker demand following a price hike and a more-than-2.5% share decline, while not establishing the eventual impact on results.	Reports of at least 15% component-order cuts amid weaker demand following a price hike and a more-than-2.5% share decline
7	0	bear	True	The evidence states that MACD was 3.43 and the signal line was 3.87, so MACD was below its signal line, which can be a bearish momentum signal.	MACD was 3.43 and the signal line was 3.87, so MACD was below its signal line, which can be a bearish momentum signal

```
4                          The evidence says the first-generation Apple TV 4K
was marked vintage four days before a new model, but does not establish that the
timing could renew interest in the product.
5                          The evidence confirms the stated revenue, net income,
and diluted EPS for the quarter, while providing no year-over-year comparison,
consistent with the claim's qualification.
6  The reports describe at least 15% cuts to iPhone 18 component orders amid
weaker-than-expected demand attributed to a price hike and a share decline
exceeding 2.5%, but do not establish the eventu…
7                     The evidence states that MACD was 3.43 and the signal line
was 3.87, confirming that MACD was below its signal line; calling this
potentially bearish is a qualified interpretation.

                                      claim
0  Apple's latest quarter demonstrates substantial earnings power, with $109.42
billion in revenue, $29.79 billion in net income, and diluted EPS of $2.02.
These results provide a strong financial ba…
1                                              The 14-day RSI of 56.1 is modestly
above the midpoint, consistent with a mildly constructive near-term technical
backdrop rather than an overbought reading.
2  Reports that Apple cut iPhone 18 component orders by at least 15% amid
weaker-than-expected demand following a price hike point to near-term sales risk
for a key product line; shares also fell mor…
3
MACD was below its signal line (3.43 versus 3.87), a potentially bearish
momentum signal for the shares.
4  Apple's TV lineup may have a product-refresh catalyst: the first-generation
Apple TV 4K was marked vintage four days before an expected successor, creating
an opportunity to renew interest in the …
5  Apple's latest quarter demonstrates substantial earnings power, with $109.42
billion in revenue, $29.79 billion in net income, and diluted EPS of $2.02.
These results provide a strong financial ba…
6  Reports that Apple cut iPhone 18 component orders by at least 15% amid
weaker-than-expected demand following a price hike point to near-term sales risk
for a key product line; shares also fell mor…
7
MACD was below its signal line (3.43 versus 3.87), a potentially bearish
momentum signal for the shares.
```

The memo. The verdict comes first, then the diagnostics, then the supported claims with their evidence ids.

[19]:
```
print(plain(result.memo))
```

```
Verdict: bear. Reported iPhone component-order cuts amid weaker demand and the
sharp share-price reaction create a meaningful near-term risk, outweighing
strong quarterly results that lack evidence of growth; the MACD signal adds only
modest support.
```

```
Diagnostics: 1 supported bull claim(s), 2 supported bear claim(s) after 2
round(s); counts informed nothing.
- [bull] Apple's latest quarter demonstrates substantial earnings power, with
$109.42 billion in revenue, $29.79 billion in net income, and diluted EPS of
$2.02. These results provide a strong financial base for the bullish case, even
though the evidence does not establish year-over-year growth. (evidence:
a95cb4269f3f, 9095a31eebd3, 2cba44c29b53)
- [bear] Reports that Apple cut iPhone 18 component orders by at least 15% amid
weaker-than-expected demand following a price hike point to near-term sales risk
for a key product line; shares also fell more than 2.5% after the report. This
is a bearish signal, though the reports do not establish the eventual impact on
results. (evidence: 2a3bab345bce, 33fcd8e975b5)
- [bear] MACD was below its signal line (3.43 versus 3.87), a potentially
bearish momentum signal for the shares. (evidence: 16572aad6cd1)
```

Self-reflection. At the end, the committee scores its own work: its confidence is the share of final claims that were supported, and its open gaps list what it went without (a missing source, or claims that were sent back). This becomes the memory note for the next run (section 6).

We also keep every model call with its stage, tokens, and time, so we can see what the loop cost.

[20]:
```
print("Confidence:", round(result.note.confidence, 2))
print("Open gaps: ", result.note.open_gaps or "none")
print("Lesson:    ", plain(result.note.lesson))
print()
calls = pd.DataFrame(
    [
        {"stage": c.stage, "tokens": c.prompt_tokens + c.completion_tokens,␣
 ↪"ms": c.latency_ms}
        for c in result.model_calls
    ]
)
if calls.empty:
    print("No model calls (rule-based run).")
else:
    display(
        calls.groupby("stage").agg(
            calls=("tokens", "size"), tokens=("tokens", "sum"), total_ms=("ms",␣
 ↪"sum")
        )
    )
```

```
Confidence: 0.75
Open gaps:  ['1 claim(s) sent back as unsupported']
Lesson:     Verdict bear after 2 round(s): Reported iPhone component-order cuts
amid weaker demand and the sharp share-price reaction create a meaningful near-
term risk, outweighing strong quarterly results that lack evidence of growth;
the MACD signal adds only modest support.
```

```
           calls  tokens  total_ms
stage
bear           1    1218      4311
bull           2    2750      9179
evaluator      8    3469     21485
verdict        1     932      3102
```

A controlled example of one full review cycle

A live run does not always reject a claim, so here is a small controlled example where we know a rejection will happen. Instead of OpenAI, we plug in a **scripted client** that gives fixed answers in the same format the model would. This is the same trick our tests use. Everything else (the loop, the two evaluator gates, the rewrite, the verdict) is the real code.

The script: the Bull analyst cites real evidence. The Bear analyst's first claim cites an evidence id that does not exist, so the rules gate rejects it without even asking the model. The Bear analyst then gets the rejection reason back and writes a replacement that cites real evidence, and round 2 accepts it.

This is not a live result. The answers are written by us to show the mechanism.

[21]:
```
from types import SimpleNamespace

from investment_committee.agents.analyst import DraftedClaim, DraftedClaims
from investment_committee.agents.evaluator import ClaimVerdict
from investment_committee.config import ENV_VARS, Settings
from investment_committee.workflows.evaluator_optimizer import␣
 ↪CommitteeVerdict, run_committee


class ScriptedClient:
    """Stands in for the OpenAI client and returns our fixed answers in order.
 ↪"""

    def __init__(self, drafts, verdicts, decisions):
        self.answers = {DraftedClaims: drafts, ClaimVerdict: verdicts,␣
 ↪CommitteeVerdict: decisions}
        self.responses = self

    def parse(self, *, model, instructions, input, text_format):
        answer = self.answers[text_format].pop(0)
        usage = SimpleNamespace(input_tokens=0, output_tokens=0)
        return SimpleNamespace(output_parsed=answer, model="scripted",␣
 ↪usage=usage)


def one_claim(evidence_id, text):
```

```
    return DraftedClaims(claims=[DraftedClaim(text=text,␣
 ↪evidence_ids=[evidence_id])])


script = ScriptedClient(
    drafts=[
        one_claim("ev-001", "Revenue grew 4 percent from a year earlier."),  #␣
 ↪Bull, round 1
        one_claim("ev-999", "Profit collapsed this quarter."),  # Bear, round 1:
 ↪ no such id
        one_claim("ev-002", "EPS beat consensus by only 4 cents."),  # Bear␣
 ↪rewrite, round 2
    ],
    verdicts=[
        ClaimVerdict(supported=True, reason="The revenue row says up 4 percent.
 ↪"),
        ClaimVerdict(supported=True, reason="Still matches the revenue row."),
        ClaimVerdict(supported=True, reason="EPS was 1.42 against 1.38."),
    ],
    decisions=[CommitteeVerdict(side="bull", reason="Growth is clear; the EPS␣
 ↪beat is small.")],
)

sample_pool = [Evidence(**item) for item in SAMPLE_EVIDENCE["evidence"]]
scripted_settings = Settings(
    values={name: "scripted" if name == "OPENAI_API_KEY" else None for name in␣
 ↪ENV_VARS}
)
controlled = run_committee(
    Run(symbol="AAPL", with_review=True), sample_pool, scripted_settings,␣
 ↪client=script
)

table(
    [
        {
            "revision": c.revision,
            "side": c.side,
            "supported": c.supported,
            "evaluator reason": c.reason,
            "evidence": ", ".join(c.evidence_ids),
            "claim": c.text,
        }
        for c in controlled.all_claims
    ]
)
```

[21]:
```
   revision  side  supported                        evaluator reason evidence  \
0         0  bull       True      The revenue row says up 4 percent.   ev-001
1         0  bear      False  cites evidence not in the pool: ev-999   ev-999
2         0  bull       True          Still matches the revenue row.   ev-001
3         1  bear       True              EPS was 1.42 against 1.38.   ev-002

                                      claim
0  Revenue grew 4 percent from a year earlier.
1              Profit collapsed this quarter.
2  Revenue grew 4 percent from a year earlier.
3          EPS beat consensus by only 4 cents.
```

Round 1 rejected the Bear claim with the reason "cites evidence not in the pool: ev-999". In round 2 the Bear analyst replaced it with a claim that cites `ev-002`, the evaluator accepted it, and the kept Bull claim was checked again. Then the loop stopped because nothing was left unsupported.

[22]:
```
print("Rounds:", controlled.rounds, "| verdict:", controlled.verdict)
print(controlled.memo)
```

```
Rounds: 2 | verdict: bull
Verdict: bull. Growth is clear; the EPS beat is small.
Diagnostics: 1 supported bull claim(s), 1 supported bear claim(s) after 2
round(s); counts informed nothing.
- [bull] Revenue grew 4 percent from a year earlier. (evidence: ev-001)
- [bear] EPS beat consensus by only 4 cents. (evidence: ev-002)
```

6. Memory across runs

Rubric item: the agent learns across runs.

After every committee run, its memory note (confidence, open gaps, lesson) is saved in SQLite. When the next run for the same stock starts, `run_research` loads the latest note and gives it to the planner.

To make the effect easy to see, we use a different stock and pretend the market tool is down in the first run. The committee should notice that it had no market evidence, and the second run's planner should move market from the last step to the first.

[23]:
```
from investment_committee.storage import load_latest_note

MEMORY_SYMBOL = "MSFT"
TOOLS_NO_MARKET = {**TOOLS, "market": lambda run: []}

with closing(open_db(DB_PATH)) as conn:
    print("Note before run 1:", load_latest_note(conn, MEMORY_SYMBOL))

first = run_research(
    MEMORY_SYMBOL, gatherers=TOOLS_NO_MARKET, database_path=DB_PATH,␣
 ↪planner_provider="rules"
```

```
)
print("Run 1 plan:", [task.source for task in first.plan.tasks])

with closing(open_db(DB_PATH)) as conn:
    first_committee = run_committee_from_storage(
        conn, first.run.run_id, settings, provider=PROVIDER
    )
    saved_note = load_latest_note(conn, MEMORY_SYMBOL)

print("Run 1 verdict:", first_committee.verdict)
print("Saved note -> confidence:", round(saved_note.confidence, 2))
print("              open gaps:", saved_note.open_gaps)
```

```
Note before run 1: None

Run 1 plan: ['earnings', 'news', 'market']

Run 1 verdict: bull
Saved note -> confidence: 1.0
              open gaps: ['no market evidence in the pool']
```

Now the second run. Nothing is passed in by hand: `run_research` reads the note from the database by itself.

[24]:
```
second = run_research(
    MEMORY_SYMBOL, gatherers=TOOLS, database_path=DB_PATH,␣
 ↪planner_provider="rules"
)
print("Lesson the planner read:", plain(second.plan.memory_lesson))
display(
    table(
        [
            {"step": n, "run 1": a.source, "run 2": b.source, "run 2 reason": b.
 ↪reason}
            for n, (a, b) in enumerate(
                zip(first.plan.tasks, second.plan.tasks, strict=True), start=1
            )
        ]
    )
)
```

```
Lesson the planner read: Verdict bull after 2 round(s): Reported quarterly
revenue of $90.01 billion, net income of $35.77 billion, and EPS of $4.81
provide concrete positive evidence, while the AI launch adds potential upside;
the bearish claim identifies costs and competition but does not quantify their
impact.

   step     run 1     run 2  \
0     1  earnings    market
1     2      news  earnings
```

```
2     3    market      news

                                                run 2 reason
0  Revisit the market gap reported by the previous run.
1              Collect the latest quarterly performance.
2    Collect recent events that may affect the company.
```

Market moved from step 3 to step 1, and its reason now says it is revisiting the gap from the last run. Nobody changed the code or the settings between the two runs; the only difference is the note the first run left behind.

The OpenAI planner gets the same note inside its prompt, so it can use it too:

[25]:
```
model_with_memory = plan_research(
    MEMORY_SYMBOL, second.run.run_id, provider="openai",␣
 ↪previous_note=saved_note, settings=settings
)
print("Planner used:", model_with_memory.provider_used)
plan_table(model_with_memory.plan)
```

```
Planner used: openai
```

[25]:
```
    step  source  \
0     1  market
1     2    news

reason
0  Fill the identified gap with current price, valuation, and performance
evidence.
1     Check recent developments around the AI launch and competitive or cost
risks.
```

7. Evaluation and iteration: with vs without review

Question: does the evaluator loop make the committee's final claims better supported?

How we test it fairly. For each stock we gather evidence **once**. Then we run the committee twice on that same saved evidence: once with the review loop and once without it (draft, judge once, decide). Both runs point to the same parent run in SQLite, so the evidence is identical and the review loop is the only setting we change. Each run still writes its own first draft, so the drafts can differ too (see the limits below).

7.1 A controlled comparison (scripted, offline)

First the same experiment with the scripted client from section 5, so the difference between the arms is easy to see and the result is the same every time. Both arms get the same scripted first drafts: a good Bull claim and a Bear claim that cites a missing id. With review, the Bear claim is sent back and rewritten. Without review, it stays rejected. **This shows how the experiment works, not whether the loop helps:** we wrote these answers ourselves.

[26]:
```
from investment_committee.evaluation.harness import plot_support_comparison,␣
 ↪run_review_comparison
from investment_committee.evaluation.metrics import comparison_rows, score_run
from investment_committee.storage import save_run_with_evidence

with_review_answers = dict(
    drafts=[
        one_claim("ev-001", "Revenue grew 4 percent from a year earlier."),
        one_claim("ev-999", "Profit collapsed this quarter."),
        one_claim("ev-002", "EPS beat consensus by only 4 cents."),
    ],
    verdicts=[ClaimVerdict(supported=True, reason="Matches the cited row.")] *␣
 ↪3,
    decisions=[CommitteeVerdict(side="none", reason="Each side has one grounded␣
 ↪point.")],
)
without_review_answers = dict(
    drafts=[
        one_claim("ev-001", "Revenue grew 4 percent from a year earlier."),
        one_claim("ev-999", "Profit collapsed this quarter."),
    ],
    verdicts=[ClaimVerdict(supported=True, reason="Matches the cited row.")],
    decisions=[CommitteeVerdict(side="bull", reason="Only the Bull claim␣
 ↪survived.")],
)
# run_review_comparison runs the with-review arm first, then the other one.
script = ScriptedClient(
    drafts=with_review_answers["drafts"] + without_review_answers["drafts"],
    verdicts=with_review_answers["verdicts"] +␣
 ↪without_review_answers["verdicts"],
    decisions=with_review_answers["decisions"] +␣
 ↪without_review_answers["decisions"],
)

controlled_conn = open_db(":memory:")
controlled_parent = Run(**SAMPLE_EVIDENCE["run"])
save_run_with_evidence(controlled_conn, controlled_parent, sample_pool)
controlled_ab = run_review_comparison(
    controlled_conn, controlled_parent.run_id, scripted_settings, client=script
)
controlled_ids = [controlled_ab.with_review.run.run_id, controlled_ab.
 ↪without_review.run.run_id]
table(comparison_rows(controlled_conn, controlled_ids))
```

[26]:

	symbol	review loop	verdict	rounds	supported claims	rejected drafts	\
0	AAPL	on	none	2	2	1	
1	AAPL	off	bull	1	1	1	

	model	calls	tokens	run id
0		7	0	ce08d52e39d8
1		4	0	eb52a477513b

With review, the Bear claim was rewritten and both sides reached the chair, so the verdict was "none". Without review, only the Bull claim survived, so the chair said "bull". The loop changed which arguments the decision was based on.

7.2 The live comparison

What we measure for each run: - **Final supported claims:** how many supported claims reach the verdict. The chair only reads supported claims, so this is how much grounded argument the decision is based on. - **Final supported share:** of the claims in the final slate, how many does the evidence support? - Rounds and model calls: what the loop costs. - Verdict: did the loop change the decision?

[27]:
```
EVAL_SYMBOLS = ["AAPL", "NVDA", "JPM"]

# Step 1: gather evidence once per stock (AAPL reuses the run from section 2.4).
parents = {SYMBOL: MAIN_RUN}
for symbol in EVAL_SYMBOLS:
    if symbol not in parents:
        gathered_run = run_research(
            symbol, gatherers=TOOLS, database_path=DB_PATH,␣
 ↪planner_provider="rules"
        )
        parents[symbol] = gathered_run.run.run_id

# Step 2: run the committee twice on each saved pool, with and without review.
comparisons = []
with closing(open_db(DB_PATH)) as conn:
    for symbol in EVAL_SYMBOLS:
        comparison = run_review_comparison(conn, parents[symbol], settings,␣
 ↪provider=PROVIDER)
        comparisons.append(comparison)
        print(f"{symbol}: done")
```

```
AAPL: done

NVDA: done

JPM: done
```

[28]:
```
rows = []
for comparison in comparisons:
    for result in (comparison.with_review, comparison.without_review):
        final = result.claims
        supported = sum(1 for claim in final if claim.supported)
```

```
        rows.append(
            {
                "symbol": result.run.symbol,
                "review loop": "on" if result.run.with_review else "off",
                "final claims": len(final),
                "final supported": supported,
                "final supported share": round(supported / len(final), 2) if␣
 ↪final else 0.0,
                "rounds": result.rounds,
                "model calls": len(result.model_calls),
                "verdict": result.verdict,
            }
        )
final_table = table(rows)
if PROVIDER == "template":
    print("Rule-based run: the template analysts cite one real item per claim,␣
 ↪so every claim")
    print("passes and both arms look the same. The comparison needs the model␣
 ↪run.")
final_table
```

[28]:

	symbol	review loop	final claims	final supported	final supported share	rounds	model calls	verdict
0	AAPL	on	6	6	1.00	2	15	bear
1	AAPL	off	5	3	0.60	1	8	bear
2	NVDA	on	5	4	0.80	2	14	bull
3	NVDA	off	4	2	0.50	1	7	bull
4	JPM	on	3	3	1.00	1	6	bull
5	JPM	off	3	2	0.67	1	6	bear

[29]:
```
columns = ["final supported", "final supported share", "model calls", "rounds"]
summary = final_table.groupby("review loop")[columns].mean()
summary.round(2)
```

[29]:

review loop	final supported	final supported share	model calls	rounds
off	2.33	0.59	7.00	1.00
on	4.33	0.93	11.67	1.67

The stored scorecard. The same runs, read back from SQLite with `comparison_rows`. Here "supported claims" and "rejected drafts" count every draft the evaluator judged, including drafts

the loop later replaced, so they show how much work the loop did.

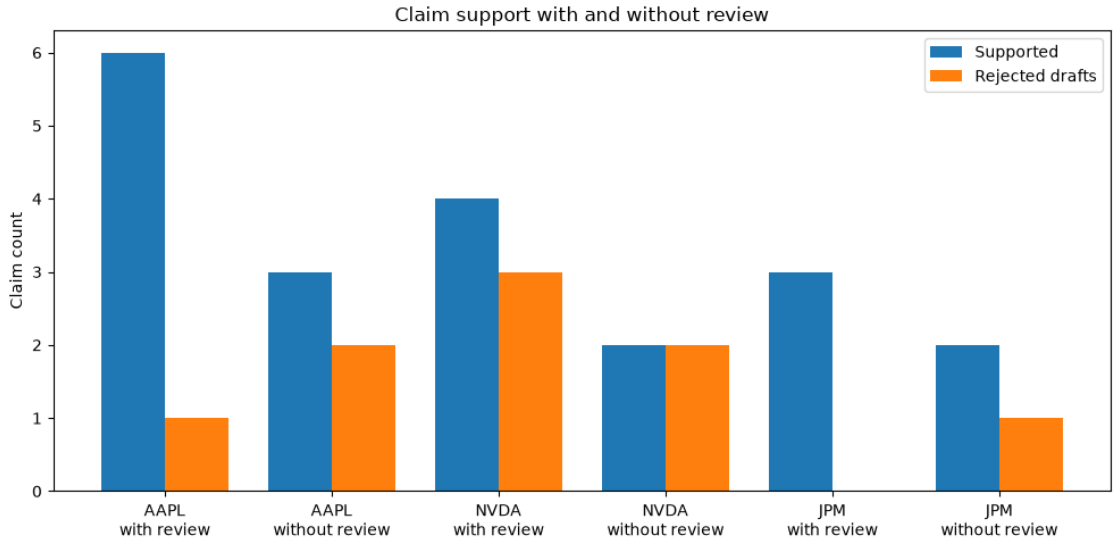
[30]:
```
run_ids = []
for comparison in comparisons:
    run_ids += [comparison.with_review.run.run_id, comparison.without_review.
 ↪run.run_id]

with closing(open_db(DB_PATH)) as conn:
    stored = table(comparison_rows(conn, run_ids))
    scores = [score_run(conn, run_id) for run_id in run_ids]
display(stored)

figure, axis = plot_support_comparison(scores)
```

	symbol	review loop	verdict	rounds	supported claims	rejected drafts	\
0	AAPL	on	bear	2	6	1	
1	AAPL	off	bear	1	3	2	
2	NVDA	on	bull	2	4	3	
3	NVDA	off	bull	1	2	2	
4	JPM	on	bull	1	3	0	
5	JPM	off	bear	1	2	1	

	model calls	tokens	run id
0	15	10641	7cd704d613df
1	8	5317	a100c5764290
2	14	9991	379ff8b0d678
3	7	5014	0872bb57693e
4	6	3468	7fd5c3c06133
5	6	3277	76c82de9ff8c



How to read this. Without review, the first drafts are judged once and go straight to the verdict. With review, rejected claims are sent back with the evaluator's reason and rewritten, and the rewrites are judged again. A rewrite can be rejected too, and after the second round the loop stops, so the final slate with review can still contain unsupported claims. The chair only reads the supported ones either way. The chart shows the extra work the loop does: rejected drafts are claims the evaluator sent back.

Limits of this test. Each arm writes its own first draft, so some of the difference between arms comes from the drafts and not from the loop. Three stocks is a small sample. And the same model family drafts and judges the claims, and model judges tend to favor their own writing (Panickssery et al., 2024).

8. Conclusions

These points are based on the saved run above. Live data and model answers change between runs, so the exact numbers will not repeat.

What worked

- The planner, tools, and routing worked on live data for every stock we tried. The fallbacks kept everything running when Yahoo's news feed broke and when there was no API key.
- Every claim in every memo cites evidence ids, and the evaluator gives a reason for every claim it judges, so each verdict can be traced back to the data.
- The news gate did its job on live news: it blocked six shopping-deal and streaming posts that only mentioned Apple, and let the iPhone order-cut stories through.
- Memory worked. A run with no market data left a note saying so, and the next run moved market from step 3 to step 1 by itself. The OpenAI planner read the same note and also put market first.
- In this run the review loop helped. On average 4.33 supported claims reached the verdict with review, compared with 2.33 without, and the final supported share went from 0.59 to 0.93. For JPM the verdict changed from bear to bull.

What did not work as well

- The loop is not free: it used about 12 model calls per stock instead of 7.
- The results move a lot between runs. We ran this comparison three times while building the notebook, and the gap in final supported share went from almost nothing (0.76 vs 0.75) to large (0.83 vs 0.50, and 0.93 vs 0.59 here). Three stocks per run is too small to make a strong claim.
- Each arm writes its own first draft, so part of the difference comes from the drafts and not from the loop. For JPM, the with-review arm happened to write a first draft that fully passed, so it never needed a second round.
- A rewrite can fail too. In section 5, the Bull analyst's rewrite was rejected again in round 2, and the loop stopped at its round limit.
- The evaluator only judges what the cited text says. It rejected "an RSI of 56.1 is above the midpoint" because the evidence gives the number but not the midpoint. That keeps claims honest, but it also throws out some reasonable ones.
- The gate is keyword based, so it also let in a Google product article ("Apple should be worried") as if it were Apple product news.

What this does not show

- These results are about whether the committee's claims are supported by the evidence it cites. They do **not** show that the verdicts predict stock prices or returns. We never tested that, and a bull or bear verdict here is not a trading signal.

With more time

- Share one first draft between both arms, so the loop is the only difference, and run more stocks several times each.
- Use a judge from a different model family than the analysts (Panickssery et al., 2024), or check claims with an NLI model.
- Replace the keyword classifier with a model, and add SEC filings for the earnings analyst.

9. AI tool disclosures

Each of us writes our own paragraph in the first person, following `docs/AI_DISCLOSURE.md`: which tools we used, what they did, what we changed or checked, and that the work reflects our own understanding.

Guna Pasupathy:

- Used Grammarly to update the sections of this notebook.
- Used Codex for reviewing the python code I wrote and asked for critical feedbacks. I incorporated lot of its suggestions which I missed at first, like the fallback news reader feature

Michael Valderrama. TODO(michael)

Mina Habib. TODO(mina)

References

Anthropic. (2024). *Building effective agents*. https://www.anthropic.com/research/building-effective-agents

Aroussi, R. (2026). *yfinance* (Version 1.7.0) [Computer software]. https://github.com/ranaroussi/yfinance

Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2023). *Improving factuality and reasoning in language models through multiagent debate*. International Conference on Machine Learning. https://arxiv.org/abs/2305.14325

Honnibal, M., Montani, I., Van Landeghem, S., & Boyd, A. (2020). *spaCy: Industrial-strength natural language processing in Python*. Zenodo. https://doi.org/10.5281/zenodo.1212303

NewsAPI. (n.d.). *News API*. Retrieved October 10, 2026, from https://newsapi.org

Panickssery, A., Bowman, S. R., & Feng, S. (2024). *LLM evaluators recognize and favor their own generations*. Advances in Neural Information Processing Systems. https://arxiv.org/abs/2404.13076

Xiao, Y., Sun, E., Luo, D., & Wang, W. (2024). *TradingAgents: Multi-agents LLM financial trading framework*. https://arxiv.org/abs/2412.20138

TODO(team): add the AI tool entries from section 9 and the OpenAI model entry, alphabetized.